

멀티모달 위기 상황 분석 매핑 시스템

Multimodal Crisis Event Analysis Mapping System

Abstract

To facilitate efficient disaster response, the rapid analysis of multimodal data is essential. The proposed AI-based system effectively evaluates disaster data across three distinct domains: informativeness, humanitarian impact, and damage severity. The system features an interactive map interface that enables users to submit real-time disaster reports while visualizing backend-processed historical data.

Keywords: Crisis Mapping, Disaster Response, Multimodal Data, Real-time response

1. Introduction

During large-scale disasters, traditional reporting systems are frequently overwhelmed, leaving numerous crises undocumented. While social media platforms offer a high-volume, real-time influx of ground-level data, the sheer noise and scale of this multimodal information make manual processing impractical.

Existing solutions frequently fail to balance real-time responsiveness with effective noise filtering. Satellite-based systems such as NASA's Crisis Mapping Toolkit prioritize macro-level changes over citizen-generated reports, whereas platforms like QCRI's AIDR rely on human-in-the-loop microtasking, leading to operational bottlenecks.

Additionally, recent efforts to employ large Vision-Language Models (VLMs) for data augmentation [1] have increased pipeline latency, thereby impeding timely responses.

To address these challenges, this work introduces a streamlined AI system based on a multimodal deep learning framework designed to automate crisis evaluation across three dimensions: **informativeness**, **humanitarian impact**, and **damage severity**.

2. Proposed Solutions

The proposed architecture consists of three core components: **a global map client-side visualizer**, **a server-side module**, and **a multimodal AI module**. The system handles

data from two main sources: direct citizen reports submitted through the web client and a simulated real-time stream of the historical CrisisMMD [2] dataset. To mitigate the constraints imposed by the high cost of live social media APIs (e.g., X), a scraper utility simulates real-time data flow by periodically ingesting CrisisMMD data under specific conditions. Once injected, the data passes through the pipeline, is processed by the AI module, and is subsequently saved in the database with its geospatial information for immediate projection onto the web client's interactive map.

2.1. Server-side and AI module

The server-side module utilizes the FastAPI framework for high-performance, asynchronous handling of concurrent AI inference requests. It coordinates data flow between the AI engine and a PostgreSQL database equipped with PostGIS, which supports the complex geospatial queries essential for disaster mapping. The entire backend is containerized via Docker.

The integrated AI engine serves as the analytical core, utilizing Differential Attention [2] and Gated Multimodal Fusion (GMU)[3] to align text and image features. It incorporates a CLIP-based text encoder [4] and a CLIP visual transformer. The engine executes a simultaneous multitasking classification strategy:

1. **Informativeness:** A binary filter isolating actionable situational data from ambient social media noise.

2. **Humanitarian Impact:** Categorizes content into five domains (e.g., infrastructure damage, affected individuals, etc.) to optimize resource allocation.
3. **Damage Severity:** Assesses physical impact by identifying visual structural anomalies and descriptive textual cues.

2.2. Client-side Visualizer

Developed with Next.js and Tailwind CSS, the responsive client-interface supports both user reporting and emergency response. The application renders incident reports as dynamic map markers enriched with AI-classified information. It features a comprehensive search and filter panel (querying by disaster type, needs, or keywords) and a secure portal for direct multimodal user submissions. An optimized low-overhead rendering ensures the map remains fully interactive even during high-density data influx.

3. Implementation and system demonstration

3.1. System workflow

The system workflow, as described in Figure 1, comprises modules for data ingestion, processing, storage, and visualization.

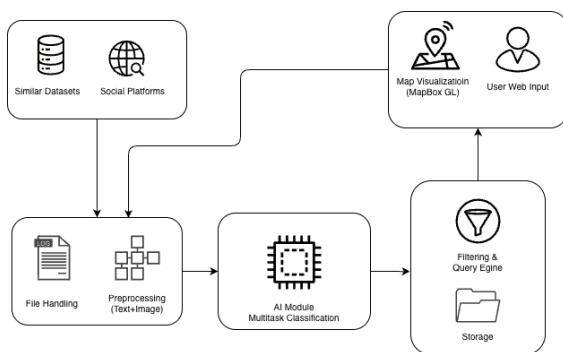


Figure 1 demonstrates the information flow and modules interactivity in the crisis mapping system

Data is collected from disaster-event datasets and social platforms, as well as user submissions containing text, images, and geolocation data. Uploaded images are stored, and corresponding URLs are generated. The data is then processed by the AI module for classification for the three tasks: informativeness, humanitarian impact, and damage severity.

Processed data, along with geographic coordinates, is stored in a PostgreSQL/PostGIS database. Non-informative reports are retained but excluded from visualization.

Upon request, the backend applies filtering based on keywords, categories, time range, and geospatial constraints. The results are returned to the client and visualized as map markers, enabling efficient transformation of raw data into actionable disaster information.

3.2. Implementation environment

The system is implemented as a modular full-stack architecture, with frameworks and tools carefully chosen to ensure scalability, reproducibility, and real-time performance.

3.2.1. Client-Side Implementation

The client-side is built with Next.js (v16.0.1), and TypeScript for a robust, scalable, maintainable architecture. Tailwind CSS (v4) is used for responsive UI design.

Geospatial visualization is handled using Mapbox GL JS, enabling real-time rendering of disaster events as interactive markers. Mapbox Geocoder supports location input, while Axios manages communication with the backend.

3.2.2. Server-Side Implementation

The backend is implemented with FastAPI, which provides asynchronous handling of concurrent requests. Data is stored in PostgreSQL with the PostGIS extension for geospatial support. SQLAlchemy and GeoAlchemy2 are used for database and spatial operations.

The backend leverages a RESTful API architecture over HTTP protocol to enable standardized, stateless communication among system components.

The AI module runs in a Docker container based on PyTorch (v2.1.2) with CUDA 11.8, deployed on an NVIDIA RTX 1080 GPU. Key libraries include Transformers, timm, torchvision, and scikit-learn.

Multimodal inference is integrated into the backend, processing text and images to generate actionable information. The model is loaded at startup using FastAPI lifespan to reduce inference latency.

3.3. User Interface Snapshots

The client interface provides an interactive platform for data submission and exploration. A global map displays disaster events as markers with the classified information, such as severity and humanitarian categories, as shown in Figure 2.

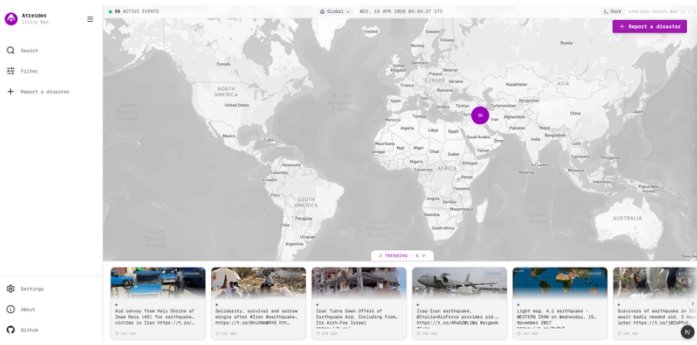


Figure 2 displaying crisis events indicated by markers with detailed images and text at the bottom of the map.

Users can access detailed information by selecting markers. A filter and search panel allows querying by keywords, geolocations, and semantic similarity.

The reporting interface enables users to submit text, images, and location data as shown in Figure 3. This information is processed in real time and displayed if informative. The interface is optimized for efficient rendering and responsiveness.

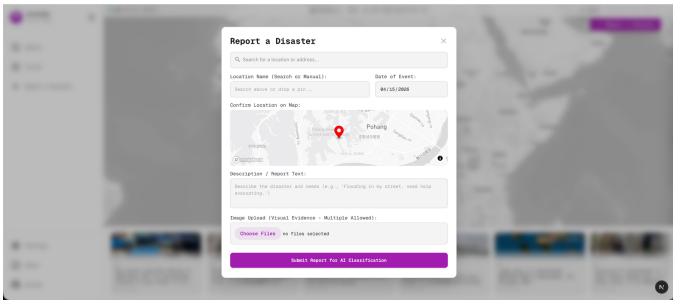


Figure 3 showing the system input for users to report an ongoing disaster with text description, image and picking location.

4. Conclusion

We proposed an integrated, end-to-end crisis-mapping platform that transforms unstructured, multimodal data into actionable disaster intelligence. By coupling a high-performance FastAPI backend, a PostGIS-enabled database, and a specialized AI classification engine, the system automates the ingestion, triage, and real-time visualization of both simulated social media streams and active citizen reports.

This full-stack architecture eliminates the bottlenecks of manual data processing, delivering an interactive geospatial dashboard that empowers emergency responders to allocate resources effectively.

Future work will focus on enhancing the system's architectural scalability, such as implementing distributed

database replication to handle massive data surges during large-scale emergencies. Additionally, we plan to broaden the backend ingestion pipeline to integrate live authoritative APIs, evolving the current system into a comprehensive, multi-source disaster management ecosystem.

Reference

- [1] Munia, N., Zhu, J., Nasraoui, O., & Imran, A. A. (2025). *Differential Attention for Multimodal Crisis Event Analysis*. arXiv preprint arXiv:2507.05165.
- [2] Alam, F., Ofli, F., & Imran, M. (2018). *CrisisMMD: Multimodal Twitter Datasets from Natural Disasters*. Proceedings of the International AAAI Conference on Web and Social Media (ICWSM).
- [3] Arevalo, J., et al. (2017). *Gated Multimodal Units for Multimodal Classification*.
- [4] Radford, A., et al. (2021). *Learning Transferable Visual Models From Natural Language Supervision*. International Conference on Machine Learning (ICML).